

DLL_703854028M

Complete Census Address Lookup Architecture

SQL Server 2025 | Windows 64-bit | Canonical synchronized release

Purpose

A single, deterministic architecture for converting high-volume U.S. Census street-range CSV files into lookup-only records, loading them without duplicates, normalizing raw addresses through SQL CLR, and returning exact, nearest, ambiguous, or not-found outcomes with centroid latitude and longitude.

Canonical naming

Component	Canonical name
Windows application	CensusAddressPreprocessor.exe
SQL CLR assembly	DLL_703854028M.dll
SQL CLR wrapper	dbo.sp000001485015000
CSV import procedure	dbo.sp000001485015001
Best-segment lookup	dbo.sp000001485015002
Main table	dbo.tblCensusAddressRangeGeocode
Completed CSV	First five numeric characters + _Complete.csv
Rejected CSV	First five numeric characters + _Rejected.csv

Package contents

- Compiled 64-bit standalone executable and SQL CLR DLL.
- C++ and C# source plus build project.
- Main-table, indexes, SQL CLR installation, duplicate-safe import, and deterministic lookup scripts.
- This consolidated architecture and implementation document.
- One README using the same canonical names.

CSV preprocessing contract

The executable processes exactly one CSV located beside it. It accepts raw Census extracted street data, legacy_Final.csv lookup data, or current_Complete.csv lookup data. It streams rows, so file size does not require loading the entire CSV into memory.

Output naming

The application scans the input filename from left to right, retains the first five numeric characters, and writes exactly 12345_Complete.csv. Example: 18089_extracted_street_data.csv becomes 18089_Complete.csv. If fewer than five numeric characters exist, processing stops without deleting the source.

Safety behavior

- The raw source is deleted only after the completed output has been finalized.
- Existing lookup files are moved to a temporary backup during replacement and restored if replacement fails.
- A rejection CSV is created only when the first invalid row occurs.
- Blank optional components remain blank and become SQL NULL; they never become zero.

Complete CSV - exact 26-column order

#	Column	#	Column
1	RowID	14	HighPrimaryNumber
2	ZIPCode	15	LowSecondaryNumber
3	StreetName	16	HighSecondaryNumber
4	StreetSoundex	17	LowAlphaSuffix
5	StreetNumericKey	18	HighAlphaSuffix
6	StreetNumericClass	19	LowFractionValue
7	PrefixDirectional	20	HighFractionValue
8	StreetSuffix	21	DerivedParity
9	SuffixDirectional	22	CentroidLatitude
10	HouseNumberType	23	CentroidLongitude
11	LowHouseNumber	24	StreetCoreName
12	HighHouseNumber	25	StreetTokenPrefixKey
13	LowPrimaryNumber	26	StreetTokenPhoneticKey

House-number representation

The complete normalized low and high endpoint strings are authoritative. Derived fields accelerate safe numeric comparisons but never overwrite the source meaning.

Type	Meaning	Lookup behavior
1	Numeric (44)	Exact range, parity, and nearest comparison allowed
2	Hyphenated (44-12)	Primary and secondary components retained
3	Alphanumeric (44A)	Numeric primary and alphabetic suffix retained
4	Fractional (44 1/2)	Primary and fractional value retained
5	Complex or mixed	Exact normalized endpoint text only; no unsafe nearest fallback

Street normalization and lookup keys

- Ordinal words are normalized to numeric forms such as SEVENTH to 7TH.
- Recognized directional and suffix values are standardized only in valid component positions.
- StreetCoreName removes recognized directional/suffix components.
- StreetTokenPrefixKey stores up to the first three characters of every core token.
- StreetTokenPhoneticKey stores one Soundex value per core token.
- Whole-street Soundex remains the last fuzzy fallback, not the only street lookup method.

Directional or suffix errors

A missing or incorrect directional/suffix does not automatically eliminate an otherwise strong address-number and street-core match. It adds a deterministic penalty. For example, 44 North 7th Avenue can still reach a row stored as 44 7th Avenue, while a fully exact candidate remains preferred.

Deterministic best-segment rule

Every candidate is evaluated by the same fixed hierarchy. Lower scores are better. No random ordering, query-plan ordering, or RowID tie-break is allowed.

Priority	Candidate evidence	Effect
1	Exact normalized full street	Strongest street tier
2	Exact core street	Fallback with component penalties
3	Numeric street key	Numeric/ordinal fallback
4	Per-token prefix key	Controlled spelling fallback
5	Per-token phonetic key	Token phonetic fallback
6	Whole-street Soundex	Broadest street fallback
House	In-range + compatible parity	Preferred segment
House	Outside numeric range	Nearest distance penalty
Complex	Exact endpoint text	Required for Type 5

Outcomes

Outcome	Meaning
EXACT	One unique lowest score with exact/in-range evidence
NEAREST	One unique lowest score selected by safe numeric distance
AMBIGUOUS	Two or more candidates share the lowest score; no ID or coordinates returned
NOT_FOUND	No eligible candidate remains

Why ambiguity is explicit

Silently choosing the smallest RowID would be deterministic but not trustworthy. The procedure therefore returns AMBIGUOUS and NULL coordinates whenever the evidence does not uniquely identify one segment.

Duplicate-safe import

RowID is excluded because it can restart in each CSV. Derived matching keys are excluded because they describe a segment but do not define a different physical segment.

Authoritative duplicate key

Field	Reason
ZIPCode	Constrains the postal geography
StreetName	Uses the complete normalized street identity
LowHouseNumber	Preserves the authoritative low endpoint
HighHouseNumber	Preserves the authoritative high endpoint
CentroidLatitude	Separates physically different segment centroids
CentroidLongitude	Separates physically different segment centroids

Three-layer protection

- A unique staging index with IGNORE_DUP_KEY removes duplicates inside one CSV.
- The TOP (10000) import uses NOT EXISTS against the main table.
- A unique main-table segment index prevents concurrent sessions from inserting the same segment.

Import performance

The batched insert limits transaction and logging pressure. The unique composite index adds write cost, but removes later duplicate-cleanup work and makes correctness independent of import timing. Expected net impact is workload-dependent; validate using representative multi-million-row files and SQL Server wait/elapsed metrics.

SQL Server 2025 table and indexes

The canonical main table is memory optimized with SCHEMA_AND_DATA durability. Its RowID primary key is a nonclustered hash index with BUCKET_COUNT = 16,777,216. Range and street searches use nonclustered indexes because ordered/range access is required.

Index	Purpose
PK_tblCensusAddressRangeGeocode	RowID point lookup; hash bucket count 16,777,216
UX_tblCensusAddressRangeGeocode_Segment	Concurrency-safe duplicate prevention
IX_tblCensusAddressRangeGeocode_Numeric	ZIP + numeric street + house-range lookup
IX_tblCensusAddressRangeGeocode_Name	ZIP + normalized street lookup
IX_tblCensusAddressRangeGeocode_Soundex	ZIP + whole-street Soundex fallback

Native-procedure boundary

SQL CLR procedures cannot be invoked directly from a natively compiled stored procedure. Call `dbo.sp000001485015000` from an interpreted wrapper/orchestrator, then pass its normalized outputs to native-compatible lookup kernels when required.

Installation order

- Run `DLL_703854028M_Main_Table_And_Indexes.sql`.
- Copy `DLL_703854028M.dll` to `C:\Phalanx\DLL_703854028M.dll`.
- Run `DLL_703854028M_Install_SQLCLR_SQL_Server_2025.sql`.
- Run `DLL_703854028M_Import_Complete_CSV_No_Duplicates.sql`.
- Run `DLL_703854028M_Deterministic_Best_Segment_Lookup.sql`.
- Replace `CHANGE_TO_PHALANX_DATABASE` in every SQL file before execution.

Operational validation

- Confirm every output filename matches `12345_Complete.csv`.
- Verify the completed CSV has exactly 26 headers in the documented order.
- Import the same file twice and confirm the second run inserts zero rows.
- Test numeric, hyphenated, alphanumeric, fractional, and Type 5 complex endpoints.
- Create an intentional equal-score tie and confirm `AMBIGUOUS` returns `NULL` ID/coordinates.
- Confirm a nearest result is never produced for a Type 5 complex endpoint.